CLOUD NATIVE DATA ENGINEERING

ASSIGNMENT 2

Design & Deploy: Relational Modeling, SQL Mastery & Cloud Database Implementation

Assignment Objectives

- **Relational Data Modeling:** Design and document conceptual and logical relational data models.
- **Dataset Acquisition:** Acquire or generate a realistic domain dataset with at least 5,000 rows.
- **Normalization (1NF–3NF):** Identify insertion, deletion, and update anomalies in flat data and normalize to 3NF.
- **Cloud Database Deployment:** Provision an AWS RDS MySQL database and load normalized relational data.
- **Analytical SQL:** Execute 8 business SQL queries leveraging joins, aggregations, CTEs, and window functions.
- **Query Optimization:** Profile and optimize query performance using execution plans (EXPLAIN / EXPLAIN ANALYZE) and indexing.
- **Cloud Cost Awareness:** Inspect cloud billing, calculate production 24/7 run-rate costs, and execute resource teardown.

Software and Account Setup

- **AWS Account:** An active AWS account under the AWS Free Tier. You will use AWS throughout the course.
- **SQL Client:** MySQL Workbench, DBeaver, or another SQL IDE / GUI client of your choice.
- **Diagramming Tool:** draw.io, Lucidchart, or MySQL Workbench EER Modeler for ER diagram generation.
- **Python Environment:** Python 3.10+ with pandas and a database connector (pymysql or SQLAlchemy), plus optional synthetic generation libraries (e.g., Faker) if generating data.

> **[DOMAIN CONTINUITY]** Use the same domain you designed in Assignment 1. Your entities, data sources, and schema should evolve directly from your Assignment 1 architecture. You will continue extending this system in Assignment 3.

1. Dataset Acquisition: Generate or Download

You must acquire a single flat, denormalized dataset (wide table) representing transactions and entities for your chosen domain. This flat table must reflect realistic operational data with natural anomalies (repeated customer names and addresses, inconsistent casing, duplicate groups, and null values) that you will normalize in Part 2.

Acquisition Options: Choose ONE of the two acquisition options below:

- **Option A — Generate Synthetic Data:** Write a Python script (using libraries such as Faker, NumPy/pandas, or custom logic) to generate realistic records for your domain. Intentionally introduce denormalization and anomalies suitable for normalization. Submit your Python generation script.
- **Option B — Download Public Data:** Download an existing public, flat/denormalized real-world dataset relevant to your domain from Kaggle, Data.gov, AWS Open Data Registry, Google Cloud Public Datasets, or UCI Machine Learning Repository. Document the source URL, provider, license, and download date.

Specific Requirements for Part 1:

- **Minimum Volume:** The dataset must contain at least 5,000 rows (recommended 5,000–50,000 rows for smooth execution on the AWS RDS Free Tier).

- **Dataset Breadth:** The flat table must span at least 10–15 columns covering multiple business concepts (e.g., transaction details, customer attributes, merchant/item information, location/category) to provide sufficient complexity for 3NF decomposition.
- **Profiling Deliverable:** Document the chosen option, total row count, column list, observed data types, and provide a 5-row sample preview in your report.

2. Normalization (1NF → 2NF → 3NF)

Using your flat denormalized dataset (5,000+ rows), perform a rigorous normalization decomposition:

- **Data Anomalies:** Using concrete examples and actual row data from your dataset, identify and clearly explain: (1) an Insertion Anomaly, (2) a Deletion Anomaly, and (3) an Update Anomaly.
- **Normalization Progression:** Decompose your flat table step-by-step into 3NF. Show your work across three separate tabs in a submitted Excel workbook (.xlsx):
  - **Tab 1 (1NF):** Eliminate repeating groups, ensure atomic values, and identify primary / candidate keys.
  - **Tab 2 (2NF):** Eliminate partial functional dependencies (non-key attributes depending on part of a composite key).
  - **Tab 3 (3NF):** Eliminate transitive functional dependencies (non-key attributes depending on other non-key attributes).
- **Assumptions & Dependencies:** In your written report, explicitly state all functional dependencies (e.g., customer_id → customer_name, customer_email) and business assumptions applied.

3. Conceptual and Logical Data Model

- **a. Conceptual ER Diagram:** Draw a conceptual ER diagram depicting core entities and their business relationships (1:1, 1:N, M:N) without listing individual attributes.
- **b. Logical ER Diagram:** Expand the conceptual model into a full logical ER diagram detailing all tables, column names, explicit SQL data types, Primary Keys (PK), Foreign Keys (FK), NULL/NOT NULL constraints, and relationship cardinalities.
- **c. Structural Requirements:** Your logical model must satisfy the following structural requirements:
  - **Entity Count:** At least 6 distinct relational entities derived from your normalized domain data.
  - **Many-to-Many Relationship:** At least one Many-to-Many (M:N) relationship resolved using an associative / junction table with foreign keys.
  - **Recursive Hierarchy:** At least one self-referencing relationship or recursive hierarchy (e.g., employee/manager, category/parent category, or another domain-appropriate hierarchy).

4. Deploy to AWS RDS

- **a. Provision RDS Instance:** Provision an Amazon RDS MySQL instance (Free Tier: db.t3.micro or db.t4g.micro, MySQL 8.0+). Configure VPC security groups to allow inbound MySQL traffic (port 3306) from your IP address only.
- **b. Schema Creation & Data Loading:** Execute your DDL statements (CREATE TABLE, PKs, FKs, constraints) on the live RDS instance and load your normalized data. You must load at least 5,000 records across your normalized tables.
- **c. Verification Evidence:** Include two verification screenshots in your report:
  - **Screenshot 1:** AWS RDS Management Console showing your DB identifier, engine version, and status ('Available').
  - **Screenshot 2:** SQL Client connected to your live RDS endpoint showing database tables and record counts (e.g., SELECT COUNT(*) results).

5. Query Your Data

Write 8 distinct SQL queries that answer realistic business questions for your chosen domain:

- **Business Question:** Directly above each query, write the business question in 1–2 plain English sentences (e.g., 'Which restaurants had the highest order cancellation rate last month?').
- **Core SQL Constructs:** Queries must demonstrate multi-table JOINs (INNER and LEFT/RIGHT OUTER) and aggregations (GROUP BY, HAVING).
- **Advanced SQL Requirement:** At least 2 queries must use Common Table Expressions (WITH clause) or Window Functions (e.g., ROW_NUMBER(), RANK(), DENSE_RANK(), SUM() OVER(...)).
- **Execution Proof:** Provide formatted SQL code alongside screenshots of the executed queries and returned result sets from your live RDS instance.

6. Optimization, With Proof

- **a. Execution Plan Profiling:** Select your two most join-heavy or resource-intensive queries from Part 5. Run EXPLAIN (or EXPLAIN ANALYZE) against your live RDS instance and paste the full output in your report.
- **b. Index Creation & Benchmarking:** Hypothesize an index (single-column or composite) that will accelerate one of these queries. Create the index (CREATE INDEX ...), re-run EXPLAIN ANALYZE, and compare the execution plan before and after.
- **c. Empirical Analysis:** In 3–4 sentences, explain why the index improved (or did not improve) performance. Your explanation must cite exact numbers from your own execution plans (e.g., reduction in rows examined, change in scan type, or execution time in ms). Generic textbook answers will not receive credit.

7. Cost Teardown & Cloud Hygiene

- **a. Billing Screenshot:** Include a screenshot of AWS Cost Explorer or the AWS Billing Dashboard showing accumulated charges / usage for your RDS instance.
- **b. Instance Teardown:** Stop or delete your RDS instance once testing is complete to avoid unintended charges. Include a screenshot confirming the instance is stopped or deleted.
- **c. Production Cost Estimation:** In 2–3 sentences, estimate what this database would cost per month if operated 24/7 in production (including multi-AZ redundancy, backups, and storage), and suggest one architectural technique to reduce ongoing costs.

Grading Rubric

Total: 100 points | Bonus: +5 points for substantive peer review

Criterion (Points)	Exceeds Expectations	Meets Expectations	Below Expectations
Dataset Acquisition & Profiling (10)	Dataset meets ≥5,000 rows; comprehensive profiling (counts, data dictionary, sample preview); generation script or source URL is well-documented.	Dataset meets 5,000-row minimum with documented source or generation script and basic profiling.	Dataset has fewer than 5,000 rows, lacks source documentation / script, or profiling is missing.
Normalization (20)	All three anomalies clearly illustrated with student's own data; 1NF→2NF→3NF in Excel is rigorous with well-reasoned functional dependencies.	Anomalies identified from own data; 3NF progression mostly correct with minor documentation gaps.	Anomalies generic or not tied to own data; normalization incomplete, incorrect, or missing Excel tabs.
Conceptual & Logical ER Model (20)	Exceeds minimums (6+ entities, M:N resolved, recursive hierarchy) with flawless cardinality, keys, and strong domain relevance.	Meets all minimums (6+ entities, M:N via bridge table, recursive hierarchy) with correct keys and cardinalities.	Missing required structural elements (fewer than 6 entities, missing M:N or recursive relationship) or key errors.
AWS RDS Deployment & Loading (15)	RDS provisioned; schema deployed; ≥5,000 records loaded; clear screenshots of console and connected SQL client.	RDS deployed, schema loaded with required data, connection verified with required screenshots.	Missing screenshots, instance not deployed, or data not loaded into RDS.
SQL Queries (15)	All 8 queries correct and efficient, clearly mapped to business questions; advanced SQL (CTEs and window functions) used skillfully.	8 queries present and correct, answering stated business questions; meets CTE / window function requirement.	Fewer than 8 queries, incorrect logic, or missing CTE / window function requirement.
Optimization With Proof (15)	EXPLAIN / EXPLAIN ANALYZE before and after index creation shows measurable gains; empirical analysis precisely cites numbers from student's output.	EXPLAIN output before/after provided; index added; explanation grounded in execution output.	EXPLAIN output missing or faked; index not created; explanation is generic textbook text without metrics.
Cost Teardown & Hygiene (5)	Realistic 24/7 monthly production estimate; thoughtful cost-reduction strategy; teardown screenshot provided.	Reasonable cost estimate provided; teardown screenshot included.	Missing teardown screenshot or unrealistic / unsupported cost estimate.
[Bonus] Peer Review (+5)	Substantive, actionable review identifying genuine design or normalization issues in peer's logical model.	Constructive peer review meeting the 3–5 sentence requirement.	Superficial, generic review or no review submitted.

Submission Deliverables

Deliverable	Format	Description
Written Report & Verification	Single PDF or Word Document	Covers Part 1 profiling, Part 2 anomaly analysis, Part 3 ER diagrams, Part 4 deployment screenshots, Part 5 queries & result screenshots, Part 6 EXPLAIN analysis, Part 7 cost teardown, and optional peer review.
Normalization Workbook	Excel Workbook (.xlsx)	Three tabs: '1NF', '2NF', and '3NF' showing step-by-step table decomposition.
SQL Script File	.sql file	Schema DDL (CREATE TABLE, keys, constraints), Part 5 queries (each commented with its business question), and Part 6 index commands.
Dataset Code or Source Link	.py script OR URL in report	Python generation script (Option A) or source URL with license and download documentation (Option B).

> **[SECURITY REMINDER]** Never include your RDS endpoint password or AWS access keys in any submitted file. Mask credentials in all screenshots.